# ASSIGNMENT – 5

**Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?**

**Answer:**

Traditional MapReduce has several limitations:

1. Heavy disk I/O because intermediate results are written to disk.
2. Slow performance for iterative algorithms and interactive queries.
3. Complex programming model requiring multiple jobs.
4. High latency in data processing.
5. Not suitable for real-time analytics.

**Why Spark is preferred:**
Spark performs in-memory processing, supports real-time analytics, machine learning, graph processing, and provides much faster execution than MapReduce.

**Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.**

**Answer:**

Spark stores intermediate data in RAM instead of repeatedly writing it to disk. Machine learning algorithms often require multiple iterations over the same dataset. Since data remains in memory, Spark avoids expensive disk reads and writes, significantly reducing execution time compared to disk-based systems such as Hadoop MapReduce.

**Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.**

**Answer:**

```
df_clean = df.dropDuplicates(["user_id", "transaction_date"])
```

**Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.**

**Answer:**

```
from pyspark.sql.functions import avg


result = df_sales.filter(df_sales.region == "West") \
        .groupBy("product_category") \
        .agg(avg("sale_amount").alias("avg_sale_amount"))
result.show()
```

**Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.**

| **Basis** | **.na.drop()** | **.na.fill()** |
|---|---|---|
| **Effect on Data** | Reduces the number of rows in the dataset. | Keeps all rows in the dataset. |
| **Data Loss** | May result in loss of important data if many rows contain null values. | No data is lost because null values are replaced. |
| **Use Case** | Used when records with missing values are considered invalid or unusable. | Used when missing values can be substituted with meaningful values. |
| **Impact on Analysis** | Can reduce dataset size and potentially affect analysis results. | Maintains dataset completeness and consistency. |
| **Output** | Returns a new DataFrame with rows containing null values removed. | Returns a new DataFrame with null values replaced. |
| **Example** | df.na.drop() | df.na.fill({"status": "Unknown"}) |

**Example:**

```
from pyspark.sql import SparkSession
# Create Spark Session
spark = SparkSession.builder.appName("FillNullValues").getOrCreate()
# Sample Data
data = [
    (1, "Active"),
    (2, None),
    (3, "Inactive"),
    (4, None)  ]
# Create DataFrame
df = spark.createDataFrame(data, ["user_id", "status"])
# Fill null values in the status column with 'Unknown'
df_filled = df.na.fill({"status": "Unknown"})
# Display result
df_filled.show()
```

**Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.**

**Answer:**

```
from pyspark.sql.functions import count
result = df.groupBy("city") \
        .agg(count("*").alias("record_count")) \
        .filter("record_count > 100")
result.show()
```

**Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?**

**Answer:**

Spark DataFrames are immutable, meaning they cannot be modified after creation. Operations such as dropping columns, renaming columns, or filtering rows create a new DataFrame instead of changing the existing one.

**Example:**

```
new_df = df.drop("age")

new_df = new_df.withColumnRenamed("name", "customer_name")
```

**Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.**

**Answer:**

```
result = df.filter(
    (df.age >= 18) &
    (df.age <= 30) &
    (df.subscription == "Premium")
)
result.show()
```

**Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?**

**Answer:**

Null values can lead to inaccurate calculations or unexpected results during aggregation operations such as sum() and avg(). Handling nulls beforehand ensures data quality, prevents errors, and produces reliable statistical results.

**Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.**

**Answer:**

```
from pyspark.sql.functions import col
from pyspark.sql.types import TimestampType
df = df.withColumn(
    "event_time",
    col("raw_timestamp").cast(TimestampType())
).drop("raw_timestamp")
```

**Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?**

**Answer:**

A shuffle occurs when Spark redistributes data across partitions during operations like groupBy(), join(), and reduceByKey(). Data moves between executors over the network so that related records are grouped together.

It is called a **wide transformation** because data from multiple partitions is required to create the output partition. Shuffling is expensive due to network communication and disk usage.

**Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.**

**Answer:**

```
result = df.filter(
    (df.email.isNotNull()) &
    (df.username != "")
)
result.show()
```

**Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?**

**Answer:**

```
from pyspark.sql.functions import min, max, mean
result = df.agg(
    min("price").alias("min_price"),
    max("price").alias("max_price"),
    mean("price").alias("avg_price")
)
result.show()
```

**Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?**

**Answer:**

When source data contains inconsistent date formats, inferSchema=true may incorrectly identify the column type or convert some values to null. This can cause data quality issues, incorrect analysis, and failures during date-based operations. For messy datasets, it is often safer to import data as strings and perform explicit date conversions.

**Q15: Write a final processing pipeline that:**

**1. Filters out duplicates.**

**2. Fills null prices with 0.**

**3. Groups by store_id to calculate total revenue.**

**Answer:**

```
from pyspark.sql.functions import sum
result = df.dropDuplicates() \
        .na.fill({"price": 0}) \
        .groupBy("store_id") \
        .agg(sum("price").alias("total_revenue"))
result.show()
```